Learning to Probe Before Acting: A Multi-Turn RL Approach

Abstract

We study credit assignment for LLM agents that act through code over multiple turns. Standard outcome-reward RL samples one completion and broadcasts a single scalar advantage across every generated token: workable when the trajectory is one block of text, coarse when an early probe or draft changes the information available to later turns. Decisions whose effect passes through external state, tool outputs, and subsequent prompts then receive the wrong credit signal.

We propose a critic-free estimator that branches rollouts at turn boundaries. For each prompt the model samples several first-turn variants and several continuations under each variant; leaf rewards define a root Monte Carlo baseline for the prompt and a per-prefix baseline for each first-turn variant, so first-turn tokens are credited only when their prefix improves the expected reward of downstream continuations. Paired with token-summed clipped policy gradients, the estimator admits a per-turn loss decomposition that recovers the exact full-trajectory gradient while substantially reducing peak activation memory.

We instantiate the method on a two-turn code-policy task in which a first-turn probe can identify a hidden environment parameter the final policy needs, and lay out a staged ablation that separates the per-turn credit signal from branching, from multi-turn training, and from single-turn RL. Experiments are pending.

1 Introduction

Modern outcome-reward RL for LLMs samples a completion, scores it, and broadcasts a single advantage across every generated token. When the completion is one block of text, that approximation is at worst coarse: each token influences later tokens only by extending the prefix. When the trajectory contains environment probes, scratchpad programs, tool calls, and drafts whose effects reach later turns through external state, it can be wrong. An early turn that gathers information is rewarded only if the final answer happens to succeed; a final answer that succeeds despite an uninformative early turn rewards the early turn anyway. The trajectory's structure — which turns matter, and through what channel — is invisible to the gradient.

We study this in code-policy generation. The agent is given an environment interface, interacts for two chat turns, and emits a Python `policy(state)` function. Only this final policy is executed and scored; the deployed object is a program, not an LLM queried online. The first turn is free: scratchpad arithmetic, environment probes, draft policies. Useful first-turn behavior receives credit only through its effect on the second turn. The setting borrows the executable-artifact framing of FunSearch and AlphaEvolve and the broader motivation of AI-Scientist-style discovery loops [10, 11, 12], but isolates the credit-assignment problem inside one prompt rather than building a full discovery system.

The concrete instance is probe-and-adapt control of a seeded batch crystallizer. The policy sets the cooling rate; the fastest safe recipe rides just below a hidden metastable limit, and crossing the limit ruins the batch. The limit leaves no trace in the readings until it is crossed, so a first-turn program can find it only by crossing it on a throwaway batch; a second-turn policy can

then ride just below it. If the final policy succeeds, a flat reward cannot say whether the probe was informative or whether the second-turn code happened to work without it.

We propose a critic-free Monte Carlo estimator for this situation. For each prompt the model samples a small tree: several first-turn variants, and several second-turn continuations below each variant. The mean reward over the whole tree estimates the prompt's root value; the mean below one variant estimates that prefix's value. Their difference advantages the first-turn tokens; the gap between a leaf reward and its prefix value advantages the second-turn tokens. The estimator pairs with a per-turn loss decomposition for token-summed clipped surrogates: because the loss is additive over disjoint token sets when all pieces share a global normalizer, one forward/backward per turn recovers the exact full-trajectory gradient at substantially reduced peak activation memory.

We instantiate the method on a custom control task in which a first-turn probe can reveal a hidden safety limit that the final policy needs. Experiments are pending.

The contributions are:

1. a critic-free turn-level Monte Carlo baseline with per-turn advantages for sparse-reward multi-turn RL;
2. an exact per-turn loss decomposition for token-summed clipped policy-gradient objectives;
3. a probe-and-adapt benchmark and an ablation design isolating the per-turn credit signal.

2 Related work

Sparse-reward multi-turn LLM agents. Recent work treats LLM agents as sequential decision-makers rather than single completions. ArCHer uses learned turn-level values for language agents [6]. RAGEN/StarPO studies trajectory-level reinforcement learning for interactive agents [4]. Agent Lightning casts arbitrary agent executions as MDP trajectories with a credit-assignment module [13]. Other work adds explicit turn-level rewards to multi-turn PPO or GRPO [14]. These approaches address the same failure mode: a final reward alone is too coarse to say which turn mattered. Our method differs by estimating turn-level baselines from final rewards alone, without a learned critic or manually designed intermediate reward.

Monte Carlo credit assignment without critics. GRPO replaces PPO's learned critic with group-relative Monte Carlo baselines [1]. Token-summed variants such as DAPO modify the normalization for long language-model completions [2]. These methods are critic-free, but a flat group baseline still assigns the same advantage to every token in a trajectory. VinePPO addresses a related issue by re-branching from intermediate reasoning states and estimating values with Monte Carlo continuations [3]. Our estimator is the turn-level analogue: the branch points are chat/tool boundaries, and the resulting scalar advantage is broadcast to the tokens of the corresponding turn.

Executable discovery and code generation. FunSearch, AlphaEvolve, and related systems show that LLMs can search over executable programs using automated evaluators [10, 11]. Work on automatic discovery of RL algorithms broadens this view to machine-designed update rules [15]. CodeRL, RLTF, and SWE-RL use execution or rule-based feedback to train code generators [16, 17, 18]. ReVeal is closer to our setting because it uses multi-turn code generation with tool feedback and dense per-turn rewards [19]. Our setting is different in that the first turn is valuable primarily as information gathering: it may reveal a hidden environment parameter that the final policy should exploit.

Scientific agents. AI-Scientist-style systems use LLMs to generate hypotheses, write and run code, analyze results, and draft papers [12]. These systems motivate our setting, but they operate at a higher level of abstraction. We study a smaller trainable component: how to assign credit inside a multi-turn, tool-mediated interaction when only the final executable artifact receives a reward.

3 Method

3.1 Setup

The agent is a language model π_θ over chat-template trajectories. The base is `Qwen3-4B-Instruct-2507` with frozen weights W; θ parameterizes a LoRA adapter $W' = W + AB^\top$. A task $\mathcal{T}$ specifies an environment (its source runs only inside the scoring sandbox; the model receives a brief in words, §3.2), S scenarios $\{\sigma_i\}$ that fix the seed and the hidden parameters, and a per-scenario reward. The scoring oracle runs a candidate policy body in a sandboxed subprocess against each scenario and returns $R(\text{code}; \mathcal{T}) = \sum_i r_i(\text{code}; \sigma_i)$. Validation failures, errors, and timeouts on σ_i score the *error floor* $r_i = -300$, calibrated below any running-but-failing trajectory (-260 at worst) — otherwise errored rollouts ($r = 0$) outrank running-but-poor rollouts and the gradient rewards crashing.

3.2 Chat-template protocol

The model is briefed the way a control engineer would be: the goal and the physics in words, the readings it will see, the quantity it will not (the metastable limit, with its range), what the probe run is for and what it offers (`env.reset()`, `env.step(a)`, the end flag and the failure reason), how the final run is scored, and the reply format. It also receives the batch record: the full-cooling logs of two earlier batches of other lots, which showered at different levels. They are data, not code: they show what a shower looks like in the readings and that the level varies from batch to batch, and say nothing about this batch's level. It receives neither the environment's source nor any equation; the brief is reproduced in Appendix C. The probe run is optional, and what a reply is follows from its content. A reply that defines `def policy(state)` is the *controller*: the tool executes it on the scored batch from its first step, rewards it, and the task ends, on whatever turn it comes. Any other block of Python, fenced or bare, is a *probe run*: it is executed in a sandbox against a probe-run environment with the same hidden parameters as the scored batch on a different batch, and the tool returns its stdout, truncated to 2000 characters, and nothing else. The environment's source runs in a private namespace and the probe-run object is a whitelist, so the probe code has no class to instantiate and no constants to read: what it learns about the batch, it learns by running it. The last turn must be the controller. A controller that does not validate is floored; a probe run that does not parse yields a textual rejection and no reward of its own, since its credit comes only through the controller that follows it. Whether to probe at all is therefore the model's first decision, and the probe rate is a quantity we measure. A length-N trajectory is $\tau = (s, y^{(1)}, m^{(1)}, \ldots, y^{(N-1)}, m^{(N-1)}, y^{(N)})$, with completions $y^{(n)}$ alternating with tool messages $m^{(n)}$, and $R(\tau) = R(\text{code in } y^{(N)})$ the only scalar reward. The remainder of the paper specializes to $N{=}2$: at most one probe run, then the controller. Generalizing the rollout, advantages, and loss decomposition below to $N{>}2$ is straightforward (see §7).

An earlier version of the protocol (used for the runs of 2026-10-25, §5) shipped the environment's source in the prompt, a mode directive on the first line of each block and a format example. The frozen model copied the example's probe in 83 of 96 cases and, without it, built crystallizers of its own with limits of its own choosing; the brief replaced it on 2026-10-26.

3.3 Token-summed clipped surrogate

For trajectory b at position t, with importance ratio $\rho_t^b = \pi_\theta/q$ and per-token advantage A_t^b,

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{tok}}} \sum_{b,t} \mathbf{1}\{\text{model token}\} \cdot \min\big(\rho_t^b A_t^b,\ \text{clip}(\rho_t^b,\ 1-\epsilon_{\text{lo}},\ 1+\epsilon_{\text{hi}}) A_t^b\big), \tag{1}$$

with N_{tok} the count of model tokens in the global batch.[1] (1) is *additive over disjoint token subsets*: for any partition of model-token positions, the loss equals the sum of partial losses computed with the same N_{tok}. This is the property the per-turn decomposition (§3.6) exploits.

3.4 Branched rollout

For each prompt s, sample G_1 turn-1 variants in one `model.generate` call: $y_i^{(1)} \sim q(\cdot \mid s)$, $i = 1, \ldots, G_1$. For each $y_i^{(1)}$ execute the tool to obtain m_i, then sample G_2 continuations $y_{i,j}^{(2)} \sim q(\cdot \mid s, y_i^{(1)}, m_i)$ in one call. Score each leaf to obtain $\{R_{i,j}\}$. Generate-call cost per prompt is $1 + G_1$, versus $2G_1G_2$ for an unbranched rollout with the same number of leaves.

3.5 Per-turn MC value estimates and advantages

Two empirical values, both pure averages over the tree:

$$\widehat{V}_0 = \frac{1}{G_1 G_2} \sum_{i,j} R_{i,j}, \qquad \widehat{V}_1^{(i)} = \frac{1}{G_2} \sum_{j} R_{i,j}. \tag{2}$$

$\widehat{V}_0$ estimates the initial-state value; $\widehat{V}_1^{(i)}$ the value after committing to turn-1 variant $y_i^{(1)}$. Per-turn advantages, broadcast to every token of the corresponding turn:

$$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0 \qquad \text{(over } y_i^{(1)}\text{)}, \tag{3}$$

$$A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)} \qquad \text{(over } y_{i,j}^{(2)}\text{)}. \tag{4}$$

The turn-2 advantage uses the conditional mean given the turn-1 prefix rather than the global per-prompt mean: a poor turn-2 sample drawn from a strong turn-1 is recognized as poor instead of inheriting its prefix's above-average outcome. Tool-message tokens are masked via `tool_mask`; their advantage is zero. Figure 1 contrasts this scheme with the two closest critic-free / per-turn-credit baselines for multi-turn LLM RL.

3.6 Per-turn loss decomposition

The token-summed clipped surrogate (1) factors over any partition of the model-token positions into disjoint subsets that share the same global normalizer N_{tok}. Letting $\mathcal{I}_n$ index the model tokens of turn n,

$$\mathcal{L} = \sum_{n=1}^{N} \mathcal{L}_{\mathcal{I}_n}, \qquad N_{\text{tok}} = \sum_{n=1}^{N} |\mathcal{I}_n|. \tag{5}$$

Per-row averaged variants (original GRPO, BNPO, Dr.GRPO) do *not* factor this way: each row carries its own denominator, so the partial sums do not combine into the single-row loss. The token-summed form is what makes the decomposition exact.

(5) is exploited to compute $\mathcal{L}$ in N forward/backward passes rather than one. Pass n forwards $[s \,|\, y^{(1)} \,|\, m^{(1)} \,|\, \ldots \,|\, y^{(n)}]$, computes $\mathcal{L}_{\mathcal{I}_n}$, backwards, and frees. By (5) the parameter gradients accumulated across the N passes equal the single-pass gradient exactly.

[1] This matches the DAPO and CISPO recipes; per-row averaging variants (original GRPO, BNPO, Dr.GRPO) divide by per-row token counts.

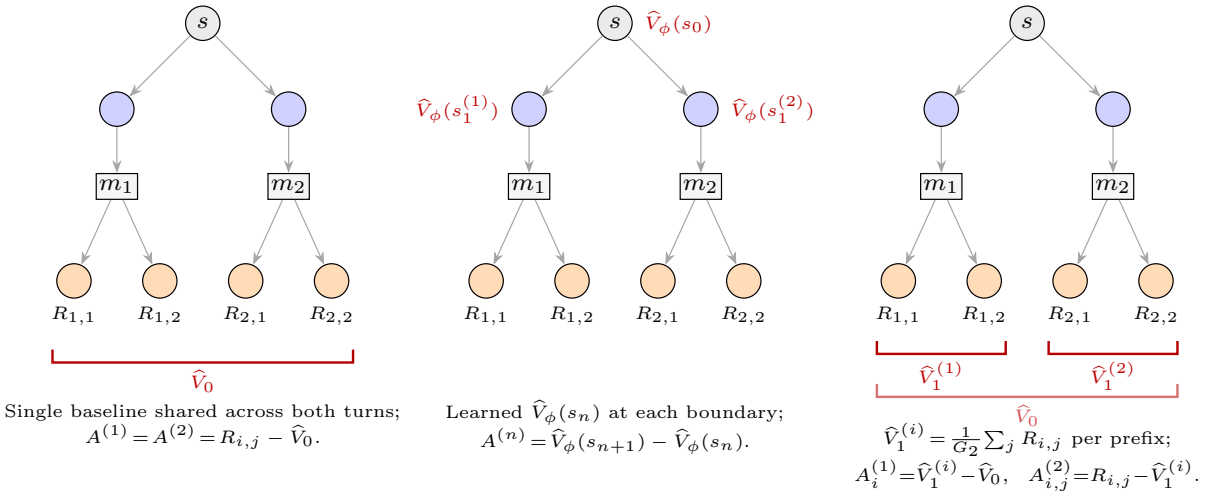


Figure 1: Comparison of credit-assignment mechanisms on a 2-turn branched rollout from prompt s with G_1=G_2=2. All three panels show the same rollout structure — turn-1 fan-out $\{y_i^{(1)}\}$, tool message m_i, turn-2 fan-out $\{y_{i,j}^{(2)}\}$, leaf rewards $R_{i,j}$ — and differ only in how the per-token baseline is computed. **(a) Flat baseline** (StarPO / RAGEN [4], the N=2 generalization of RLOO/GRPO [1] from Figure 2 of [3]): one mean $\widehat{V}_0$ over all G_1G_2 leaves, broadcast to every model token of both turns. A poor turn-2 from a strong turn-1 inherits its prefix's above-average outcome. **(b) Per-turn critic** (ArCHer [6]): a learned value head $\widehat{V}_\phi(s_n)$ provides per-state baselines at each turn boundary, with the moving-target bias issues typical of bootstrapped value learning. **(c) Per-turn MC** (ours, Arm F): the conditional mean $\widehat{V}_1^{(i)} = \frac{1}{G_2}\sum_j R_{i,j}$ over the G_2 continuations of turn-1 variant i baselines that variant's turn-1 tokens; $\widehat{V}_0$ baselines the root. Turn-2 tokens are scored against their own prefix's $\widehat{V}_1^{(i)}$, not the global mean. Group-relative within a turn, VinePPO-like across turns — i.e., VinePPO's intra-completion MC re-branching [3], applied at turn boundaries.

Why this saves memory. The dominant activation cost is the LM-head logit tensor ($|V|$ = 151,936). A single forward+backward over the full trajectory holds the saved-for-backward LM-head tensors at all $\sum_n T_n$ model-token positions simultaneously; doing one forward+backward per turn holds at most $\max_n T_n$ positions at peak, and (5) guarantees the accumulated gradient is identical. This is gradient accumulation at the turn level within a trajectory — the saving is a factor of roughly N for equal-length turns, $\approx 2\times$ at our N=2 settings. Appendix A works through the accounting.

Restrictions. The decomposition's exactness has one *conceptual* requirement and two *implementation* requirements. Conceptual: the token-summed normalizer of (1) — per-row averaged variants do not factor across token-position partitions, as discussed above. Implementation: $\beta = 0$ (a KL term to π_{ref} would require per-segment slicing of reference log-probabilities) and `num_iterations`= 1 (so cached `old_per_token_logps` need no re-segmenting). The two implementation requirements can be lifted with extra bookkeeping at no theoretical cost (§7).

4 Experimental setup

Environment. We use a seeded batch cooling crystallizer rather than a standard benchmark whose solution may already be familiar to the base model. A saturated solution holding a known mass of seed crystals is cooled from 55–65 °C to a target of 18–25 °C. The policy observes

$\{t, T, c, L, T_{\text{target}}, \text{steps_left}, \sigma\}$ (time, temperature, dissolved concentration, mean crystal size, target, and the relative supersaturation $\sigma = c/c^*(T) - 1$, reported as a reading as an in-line concentration probe would) and outputs the jacket rate as a fraction of its 1.5 °C/min limit, clipped to $[-1, 1]$, once per 20 s step for at most 600 steps. Cooling raises σ; the seeds grow and consume it. Success requires reaching the target temperature with the solution equilibrated ($\sigma \leq 0.02$). If the reported σ ever exceeds the hidden metastable limit $\sigma_{\max} \sim U[0.08, 0.25]$, a shower of new nuclei ruins the batch and the episode ends in failure. The reward is -0.3 per step, $+120$ on success and -80 on failure. Full dynamics are in Appendix B.

Why it tests turn-level credit. The time-optimal recipe rides just below $\sigma_{\max}$, but $\sigma_{\max}$ enters the dynamics only through the shower test: the readings of a batch that never showers are identical for every value of the limit. A final policy therefore cannot learn the limit in-episode, and its best blind option is to ride at the lowest limit in the range, which costs two to three times the optimal batch time at the top of the range. A first-turn probe run, which has the same actions and readings and the same hidden limit on a different batch, can find the limit the only way physics allows: cool at the jacket limit until the batch showers and keep the last safe reading. One batch brackets the limit to the rise of one step, about 0.01; from readings alone, further batches that creep up from the last safe level bring the median error to 0.004. This makes the first turn useful only through its effect on the final policy. Whether the gap between a blind controller and a limit-aware oracle is large enough to carry the study was settled with hand-written controllers before any model was run (Stage 0, §5): the best blind controller reaches 7.5% fast success over the range against 100% for the informed one, and a hand-written probe run plus controller written from the brief alone, readings only, reaches 100% fast success through the training runner (Stage 0b).

Model and training. We train LoRA rank $r = 16$ on attention projections of `Qwen3-4B-Instruct-2507`; base weights are frozen, bf16 is used, and gradient checkpointing is enabled. Unless stated otherwise, $G_1 = G_2 = 4$ and each turn is capped at 400 tokens. The loss is the token-summed clipped surrogate (1) with $\epsilon_{\text{lo}} = 0.2$, asymmetric upper clipping, $\beta = 0$, no critic, no GAE, and no context truncation across turns.

Diagnostics. We log $\widehat{V}_0$, the between-prefix and within-prefix reward standard deviations, their ratio, per-turn advantage scales, error-floor rate, and generate calls per prompt. The between/within ratio is also used diagnostically: values above one indicate that first-turn choice dominates continuation noise, where per-turn credit should help most.

5 Experiment plan

The experiments form a ladder of stages. Each stage asks one question and is a go-or-stop check for the next, so that no result is built on a step that failed. Stages 0 and 0b use hand-written controllers only; Stages 1 and 2 run the frozen base model; Stages 3 to 6 train, and correspond to arms C to F of Table 1. Throughout, *fast success* means that the final run succeeds within 1.25× the time-optimal batch time (§6), the *blind* reference is the best hand-written controller that does not know the hidden limit, and the *oracle* is the controller that does. Each subsection records the stage's question, what decides its outcome, and its status; results are filled in as stages complete, with the evidence in the dated log `docs/findings.tex`.

5.1 Stage 0: is the environment worth it?

Question. Does knowing the hidden limit buy anything that a controller without it cannot get from the final run's own readings? **Protocol.** A blind side (one hand-written controller

for every limit in the range) and an informed side (one controller per limit) search the same controller families with the same optimizer budget on the same seeds, and are scored on fresh seeds. Three gates were fixed beforehand: the informed side reaches $\geq 90\%$ fast success at every limit (G1); it beats the blind side by ≥ 40 percentage points of fast success over the range (G2); a hand-written probe run plus final run reaches $\geq 95\%$ fast success (G3). **Status.** Passed on 2026-10-24, on the environment as it then was (shower test on the mid-step supersaturation, time weight 0.1). The best blind controller rides at the lowest limit in the range and reaches 7.5% fast success (normalised time 0.50); the informed side reaches 100% at every limit; the hand-written two-turn solution reaches 100%, recovering the limit to 2×10^{-4} with the solubility curve in hand. The same gate had failed for the earlier slosh-tank environment, which was dropped. The environment changed on 2026-10-26 (σ a reading, the shower test on the reported value, time weight 0.3; Appendix B); the gate's conclusion stands, since the blind controller is unchanged and the informed one only got faster (130 against 306 steps on the 36 task batches).

5.2 Stage 0b: the hand-written solution through the training runner

Question. Does the Stage 0 solution survive the real execution path: the sandbox subprocess, the probe-run environment with the limit masked, and a probe batch different from the scoring batch? **Protocol.** A hand-written probe source is pushed through the runner's turn executor, its printed bracket is pasted into a controller template, and the controller is scored as the trainer scores it, on the 36 task-file batches plus fresh ones. Gate: fast success $\geq 95\%$. **Status.** Passed on 2026-10-25 with the Stage 0 solution (236 batches, 100% fast, probe error 2×10^{-4}). Redone on 2026-10-26 from the brief alone, readings only, against the changed environment: a three-batch probe that resets, steps and reads σ, and a feedback controller with memory that holds the reading 0.003 under the level found, give 100% fast success on 136 batches (normalised time 0.95, median probe error 0.004, at most 0.011) where the blind controller is fast on 4%. The task is solvable with exactly what the model is told.

5.3 Stage 1: the frozen model in one turn (arm A)

Question. Can the base model write a working cooling policy from the prompt alone, and how fast is it without a probe run? **Protocol.** N one-turn samples per held-out task, scored exactly as in training; fast success, normalised time and reward next to the blind and oracle references. **Decides.** The floor of the study. A rate near the blind reference shows the headroom a probe run can buy. A model that cannot write a running controller at all is a prompt problem to fix before anything else. **Status.** Done on 2026-10-25 under the protocol prompt (12 held-out batches, 8 samples each, temperature 0.8). **Result.** The code runs in 99% of samples, but the batch is ruined in 79 of 96: the model takes the top of the stated range as its target (the constant 0.25 appears in 92 of 96 policies) and cools too hard from the first step. Success 12.5%, fast success 1%, normalised time 0.075, mean reward -61, against $+89$ for the blind cautious controller and $+107$ for the oracle. The frozen model sits far below the blind reference, so the headroom is the whole scale: caution first, then the probe. **Under the brief** (2026-10-26, changed environment, rewards not comparable): 68 of 96 controllers are valid, 28 are cut off by the 600-token budget mid-code; 14 slow successes, no fast one, 27 ruined after a median of 23 steps, 24 timeouts, most of them controllers that never cool. Mean reward -160 with the floor at -300, against $+27$ blind and $+82$ oracle.

5.4 Stage 2: the frozen model with a probe run (arm B)

Question. Does the frozen model already write a useful probe run and use its output? **Protocol.** As Stage 1, with two turns under the training protocol: the final turn must be a policy, and an early policy does not end the rollout. The turn-1 mode and code are kept per sample, so the record shows which probe the model writes: none, a slow creep, or a fast one. **Decides.** If

arm B is already near the informed ceiling, RL has nothing left to teach about probing and the environment must get harder before any training run (hide the growth constant too, or add a jacket lag). If arm B is near arm A, the gap between them is the training target. **Status.** Done on 2026-10-25 under the protocol prompt. **Result.** The model used the probe run in all 96 samples, but 83 probe runs only reset the environment and print its initial state, the shape of the prompt's format example; 9 step a few times at full cooling, on an environment they construct themselves with a random limit; none brackets the limit. The final policies paste numbers from the probe batch into code that runs on a different batch (93 of 96) and all contain the constant 0.25: success 3 of 96, no fast success, normalised time 0.013, mean reward −76, below arm A. The probe run is a ritual, not an experiment, and the risk that the frozen model already writes the one-batch probe did not materialise. **Under the brief** (2026-10-26): the ritual is gone. 87 of 96 probes step the environment, 61 cool at the jacket limit, 92 check the end flag or the failure reason, 30 run several batches, none builds its own crystallizer; the experiment has the right shape in nearly every sample. But 44 probes are cut off by the 600-token budget before they can run, 22 crash on reading the end flag as a state key, and only 5 outputs hold the limit to within 0.01. Only 7 controllers reuse a number the probe printed; when the probe failed, several invented its results. Success 24 of 96 (2 fast, the fast one on invented results), 24 ruined after a median of 20 steps at full cooling towards a setpoint of 0.18–0.20 with no margin, 33 timeouts, mean reward −116 against +27 blind and +82 oracle. The training target is now concrete: get the number out of the probe and leave a margin.

Example ablation (2026-10-26, config key `prompt.example`). The same protocol with three versions of the prompt's format example, 96 samples each: the original (*full*: reset the environment and print the state), the same two blocks with a comment in place of the probe code (*format*), and no example (*none*). Full replicated the result above: 80 of 96 probe runs copy the example. None removed the probe run altogether: the model answered with a policy in all 96 first turns, 59 of them without the required fence, so they were rejected. Format removed the copying (31 of 96) and produced experiments: 61 probe runs say they will find the limit and 44 step an environment in a loop and check the end flag, but every one constructs a fresh crystallizer instead of using the one provided, 39 with a limit of the model's own choosing, so the experiment can only confirm a guess; 5 heat instead of cool; and no final policy takes a number from its probe output, 92 of 96 keep the constant 0.25 (success 7 of 96, fast 1). The probes are three times longer and 17 ran out of the 600-token turn budget. The format example is the default from here on. What the frozen model lacks is not the idea of an experiment but two facts about the given environment: that the probe must run on it rather than on a copy, and that the number found there must become the setpoint.

5.5 Stage 3: single-turn GRPO (arm C)

Question. Does RL fine-tuning help without a probe run? **Decides.** The ceiling is the blind optimum, so the stage checks the training plumbing and the reward rather than the thesis. It also shows whether the time weight tempts the model to gamble on low-limit batches; the weight was fixed at 0.3 on 2026-10-26 (a good probe is then worth +53 over the blind controller, against +18 at 0.1, while a ruined batch still costs 114 more than the blind controller earns). **Status.** Pending. **Result.** *To be filled.*

5.6 Stage 4: flat multi-turn training (arm D)

Question. Does multi-turn training with a trajectory-level advantage beat single-turn training? **Decides.** The baseline the headline claim is measured against. **Status.** Pending. **Result.** *To be filled.*

5.7 Stage 5: branched rollouts with a flat advantage (arm E)

Question. Does the rollout tree alone help, independent of the per-turn credit? **Decides.** Separates the effect of more leaves per generate call from the effect of the per-turn baseline. **Status.** Pending. **Result.** *To be filled.*

5.8 Stage 6: branched rollouts with per-turn baselines, ours (arm F)

Question. Does the per-turn baseline help? **Decides.** F above D at matched leaves is the minimum convincing result. F equal to E while both beat D means the gain comes from branching, not from credit. C to F not beating A and B means the protocol is better framed as prompting than as training. **Status.** Pending. **Result.** *To be filled.*

Stages 3 to 6 share the protocol of §6: at least three seeds per arm, median and interquartile range at the best checkpoint, curves against gradient steps and against wall-clock, and a paired Wilcoxon test with effect size.

6 Evaluating the per-turn credit assignment

The substantive empirical claim is the credit-assignment contribution: hierarchical per-turn advantages (3)–(4) should outperform a flat trajectory-level advantage broadcast across both turns. Table 1 defines the six-arm ablation. The key comparisons are E vs. D for the rollout tree and F vs. E for the per-turn baseline.

Arm	Rollout structure	Advantage	What it isolates over the previous arm
A — Frozen, one-shot	N=1 inference, no training	—	Base model's prior on the code-policy interface
B — Frozen, multi-turn prompted	N=2 inference, no training	—	Does the two-turn protocol help at inference time? (Voyager-/FunSearch-style frozen-LLM use)
C — Single-turn GRPO	G flat N=1 rollouts, trained	Standard GRPO advantage	Does RL fine-tuning help at all?
D — Flat StarPO	G independent N=2 trajectories, trained	$A(\tau) = R(\tau) - \bar{R}$, broadcast to all tokens	Does multi-turn training help over single-turn? (RAGEN-style)
E — Branched, flat	$G_1 \times G_2$ tree, trained	$A_{i,j} = R_{i,j} - \bar{R}$ on all tokens	Does the tree alone help (more leaves per generate call), independent of credit?
F — Branched, hierarchical (ours)	$G_1 \times G_2$ tree, trained	$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0$ on $y_i^{(1)}$; $A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)}$ on $y_{i,j}^{(2)}$	The per-turn baseline contribution

Table 1: Ablation arms, ordered from weakest baseline (top) to ours (bottom). Trained arms (C–F) match leaf count $G = G_1 G_2 = 16$ so they see the same number of scored trajectories per step. D uses $2G$ `generate` calls per prompt; F uses $1 + G_1$. Report both equal-rollouts and equal-wall-clock comparisons. Frozen arms (A, B) use `Qwen3-4B-Instruct-2507` (§3.1) with no gradient updates, and serve as training-free references analogous to Voyager- or FunSearch-style frozen-LLM uses of the code-policy interface.

Primary metric. Held-out *fast-success* rate: the fraction of (sample, batch) pairs in which the final run succeeds within 1.25× the time-optimal batch time for that batch and limit, on 12 held-out batches whose limits are drawn from the training range. Raw success is not informative here,

since a blind cautious policy succeeds on every batch by being slow; we also report normalised time (oracle steps over own steps, zero on failure) and mean reward. A stress set with limits outside the training range, where a fixed cautious policy is furthest from optimal, is planned. Results are taken at the best checkpoint and reported as median and IQR over ≥ 3 seeds per arm, plotted against gradient steps and against wall-clock. Statistical test: paired Wilcoxon over per-seed best-checkpoint fast-success rates; report effect size, not just p. The minimum convincing comparison is F > D at matched leaves.

Diagnostic checks. We track two mechanism checks. First, when turn 1 emits a `policy`, its reward should improve faster under F than under flat multi-turn training. Second, prompts with high iteration-0 signal ratio $\widehat{\sigma}(\widehat{V}_1^{(i)})/\overline{\widehat{\sigma}(R_{i,\cdot})}$ should show the largest F–D gain; otherwise the proposed credit-assignment explanation is weak.

Failure modes. If F does not beat D, the hierarchical baseline is not useful in this domain. If F $\approx$ E while both beat D, the benefit comes from branching rather than turn-level credit. If C–F do not beat A/B, the protocol is better framed as prompting rather than RL training.

7 Limitations and future work

The implementation and experiments use $N = 2$. Extending to more turns is conceptually direct: each boundary value becomes the conditional Monte Carlo mean over the subtree below it, and the loss decomposition applies one partial loss per turn. The cost is bookkeeping and additional forward/backward passes, not a new estimator.

The method also assumes deterministic rewards given the seed. In stochastic environments, the same estimator applies but G_2 must grow to separate prefix quality from environment noise. The current implementation uses $\beta = 0$ and `num_iterations`= 1; KL penalties and cached old log-probabilities can be supported by storing the same quantities per turn segment.

A natural memory/variance extension is token subsampling within a turn. Since each turn uses one scalar advantage broadcast across its tokens, subsampling tokens is an unbiased estimator of the same token-summed gradient when N_{tok} counts only kept positions. We leave uniform and asymmetric subsampling variants to future work.

8 Conclusion

We present a critic-free, turn-level credit-assignment scheme for sparse-reward multi-turn RL. The method branches rollouts at chat-turn boundaries, uses the resulting Monte Carlo conditional means as per-turn baselines, and pairs with a per-turn loss decomposition that recovers the exact full-trajectory gradient one turn at a time.

The experiments that test this — frozen one- and two-turn references, single-turn RL, flat trajectory-level multi-turn optimization, and branched optimization with per-turn baselines — are pending. The estimator extends naturally to more turns and to stochastic rewards (Section 7); whether the per-turn signal closes the gap between a blind and an informed final policy is the headline empirical claim under test.

A Memory accounting and the trailing-window LM-head

The loss only needs logits whose successor token was emitted by the model; prompt and tool-message logits do not contribute. HF `transformers` exposes this through `logits_to_keep`: an integer keeps the last K positions, while an index tensor selects arbitrary positions.

This reduces unnecessary logits, but a single forward/backward still holds all selected LM-head outputs until backward completes. With gradient checkpointing, transformer-body activations are comparatively small; the expensive saved tensor is the LM-head output of shape $(B, K, |V|)$. Therefore, a full multi-turn backward over all model tokens scales with $K = \sum_n T_n$.

The per-turn decomposition serializes these positions. Turn n forwards the prefix through $y^{(n)}$, computes the partial token-summed loss for that turn using the global N_{tok}, backpropagates, and frees the graph. By additivity of (1), the accumulated gradient equals the single-pass gradient, while the peak LM-head tensor scales with $\max_n T_n$.

Regime	Positions through LM head	Logit tensor
Single pass, no `logits_to_keep`	$T_{\text{full}} = 1350$	≈ 13.1 GB
Single pass, selected model tokens	$\sum_n T_n = 800$	≈ 7.8 GB
Two turn-wise passes	400 per pass, freed between	≈ 3.9 GB

Table 2: Arithmetic LM-head logit sizes for $B = 16$, $|V| \approx 152{,}000$, and 4-byte logits. Total GPU memory also includes weights, optimizer state, body activations, KV cache, and allocator overhead.

The decomposition is not a new `logits_to_keep` trick; all regimes use the same primitive. The non-trivial point is that token-summed normalization makes the per-turn gradients sum exactly to the full-trajectory gradient. Per-row-averaged variants would require either a different normalizer or an approximation.

Forward-side temporaries change the binding constraint. Table 2 accounts for the LM-head logits tensor as it appears in the saved-for-backward computation graph. At production sizing on a 24 GB card, the binding allocation is not the gradient at backward but the forward itself. The softmax-stability convention in TRL holds logits in fp32 rather than bf16, doubling the byte budget over a naive estimate. Worse, the temperature scaling `logits = logits / self.temperature` is not in place: a same-shape fresh tensor is allocated before the original is freed. At the divide step *two* $(B, T, |V|)$ fp32 tensors are simultaneously live, ≈ 7.8 GiB at our production sizing. Stacked on model weights, the just-emitted turn-n KV cache from generation that has not yet been reclaimed by the allocator, and the previous turn's backward residue, this allocation crosses the 22 GiB usable budget and OOMs at step 0.

The per-turn loss decomposition addresses peak *gradient* memory but does not shrink the forward logits tensor. The remedy is row-wise chunking of the per-turn pass: split the B trajectories into sub-batches of size $b \ll B$, forward each sub-batch separately, backpropagate, and accumulate. Because the token-summed loss (1) is additive over disjoint row subsets sharing the same global normalizer N_{tok}, the chunked gradient is bit-equivalent to the single-pass gradient. The forward-time logits tensor shrinks linearly from $(B, T_n, |V|)$ to $(b, T_n, |V|)$, and the divide-step peak scales correspondingly. At $b = 4$, $B = 16$, $T = 400$ the divide-step peak drops from ≈ 7.8 GiB to ≈ 2 GiB; the cost is B/b additional LM-head kernel launches per turn (single-digit-percent wall-clock at our generation-dominated sizing). We use $b = 4$ on 24 GB L4 and run un-chunked on ≥ 40 GB cards.

B Seeded cooling crystallizer

One kilogram of solvent holds dissolved solute at concentration c (kg per kg solvent) and N monodisperse seed crystals of size L. The solubility is $c^*(T) = 0.20 + 0.006\,T + 0.0001\,T^2$ with T in °C and the relative supersaturation is $\sigma = c/c^*(T) - 1$. A batch starts saturated at $T_0 \sim U[55, 65]$ °C with a seed mass equal to 3–6% of the theoretical yield down to the target $T_{\text{target}} \sim U[18, 25]$ °C. In each 20 s step the jacket moves the temperature by the commanded

fraction of 1.5 °C/min, the crystals grow for 20 s driven by the supersaturation at the new temperature, and the shower test is applied to the supersaturation at the end of the step, the value the state reports (until 2026-10-26 the test used the mid-step value before growth, which the readings never show):

$$\frac{dL}{dt} = k_g\,\sigma^g, \qquad k_g = 1.5\times10^{-6}\,\mathrm{m/s}, \quad g = 1.5,$$
$$\frac{dc}{dt} = -\rho N\,3L^2\,\frac{dL}{dt}, \qquad \rho = 2000\,\mathrm{kg/m^3}, \tag{6}$$

with dissolution by the same law when $\sigma < 0$ and the mass transfer capped so that one step cannot cross saturation. Success requires $T \le T_{\mathrm{target}} + 0.25\,^{\circ}\mathrm{C}$ and $\sigma \le 0.02$. The episode ends in failure if the reported σ exceeds $\sigma_{\max}$ at the end of any step (a shower of new nuclei) or if the seeds dissolve below 30% of their initial size, and by timeout after 600 steps. The reward is -0.3 per step, $+120$ on success and -80 on failure.

Why the metastable limit is the hidden quantity. $\sigma_{\max}$ appears nowhere except in the shower test, so two batches with different limits produce identical readings under identical actions until one of them showers; no in-episode estimator exists. The time-optimal controller holds the reported σ at $\sigma_{\max}$: with the dynamics known, the largest cooling that keeps the end-of-step value at the limit is found by bisection each step, and it accelerates as the crystals grow (classical programmed cooling); on the 36 task batches it needs 130 steps on average. A controller that does not know the limit must ride at the bottom of the range ($\sigma \approx 0.077$), which succeeds on every batch but takes 306 steps on average, 2–3 times longer at the top of the range. The probe run finds the limit by crossing it: cooling at the jacket limit until the shower brackets $\sigma_{\max}$ to the rise of one step, about 0.01; from readings alone, two further batches that creep up from the last safe level bring the median error to 0.004 (at most 0.011), which a feedback controller with a margin of 0.003 turns into 95% of the oracle's speed. A slow single-batch creep is worse, not better: at high limits the growing crystals consume supersaturation faster than the creep adds it, and the batch equilibrates without ever reaching the limit.

Configuration.

Parameter	Symbol	Value	Status
Step length, horizon	Δt, N_{steps}	20 s, 600 steps	fixed
Metastable limit	$\sigma_{\max}$	$U[0.08, 0.25]$, one draw per task	hidden
Jacket rate limit	$\dot{T}_{\max}$	1.5 °C/min	fixed
Growth constant, order	k_g, g	1.5×10^{-6} m/s, 1.5	fixed
Seed size, loading	L_0	100 μm, 3–6% of yield	per batch
Start, target temperature	T_0, T_{target}	$U[55, 65]$, $U[18, 25]$ °C	per batch
Success tolerances	ϵ_T, ϵ_σ	0.25 °C, 0.02	fixed
Reward		-0.3 per step, $+120$, -80	fixed
Error floor		-300 per scenario	fixed

Each training task is one batch (its seed) with its own hidden limit; the probe run uses the same limit on a different batch, and the probe-run object offers `reset`, `step`, the end flag, the failure reason and the two outcome tests, the same readings as the final run and nothing else; the environment's source runs in a private namespace, so the probe code has no class to instantiate. The time-optimal number of steps for the scoring batch is stored with the task and defines the fast-success and normalised-time metrics. The two-turn protocol is useful only if a probe-aware controller can close the gap: the blind cautious controller sets the no-probe reference, the oracle sets the upper bound, and the hand-written probe run plus final run is the achievable probe-aware reference.

C The brief

The first user message of a two-turn rollout, rendered for one held-out task (the single-turn brief is the same text without the probe-run section, asking for the controller directly). Lines are wrapped for print.

```
You are the control engineer for a seeded batch cooling crystallizer: a
stirred tank of hot solution holding a known mass of small seed crystals,
cooled through a jacket whose rate you set. Your job is a Python controller
for one batch.

## Goal

Cool the batch to its target temperature as fast as possible without ruining
it.

The liquid holds less dissolved material as it cools. Whatever it cannot hold
is the supersaturation, the reading sigma: sigma = 0.10 means ten percent more
is dissolved than the liquid can hold at the current temperature.
Supersaturation makes the seed crystals grow, and growth uses it up, so at
constant temperature sigma falls back towards zero; the more crystal surface
there is, the faster. If sigma rises too far the liquid showers: a cloud of
tiny new crystals forms at once and the batch is ruined. Heating does the
opposite of cooling: it dissolves the seeds, and if they shrink below 30% of
their starting size the batch is lost as well.

Success: T <= T_target + 0.25 degC with sigma at most 0.02. The episode ends
at success, at a shower, when the seeds are lost, or after 600 steps (200
min). Reward: -0.3 per step, +120 on success, -80 if the batch is ruined.

## What you observe

`state` is a dict with keys: t (elapsed steps), T (solution temperature,
degC), c (dissolved material, kg per kg solvent), L (mean crystal size, m),
T_target (target temperature, degC), steps_left (steps remaining), sigma
(relative supersaturation, dimensionless).

`action` is a float in [-1, 1]: the jacket rate as a fraction of its 1.5
degC/min limit; negative cools, positive heats; the env clips it to [-1, 1].
One step is 20 s, so full cooling lowers T by 0.5 degC per step.

## What you do not observe

sigma_max, the level of sigma at which this batch showers. It is not in the
state, and nothing in the readings depends on it until it is crossed. It is
fixed for this task and lies between 0.08 and 0.25. How fast you can safely
cool depends on it.

## What you know from earlier batches

Batch A, quenched at full cooling from 57.7 degC (target 20.4 degC):
step    T      sigma
   1   57.2  0.010
   2   56.7  0.020
   3   56.2  0.030
   4   55.7  0.041
   5   55.2  0.051
   6   54.7  0.061
   7   54.2  0.072
   8   53.7  0.082
   9   53.2  0.092
  10   52.7  0.103
  11   52.2  0.113   showered -> batch ruined

Batch B, quenched at full cooling from 63.3 degC (target 24.3 degC):
step    T      sigma
   1   62.8  0.010
   2   62.3  0.019
   3   61.8  0.029
   4   61.3  0.039
   5   60.8  0.048
   6   60.3  0.058
```

```
   7   59.8  0.068
   8   59.3  0.078
   9   58.8  0.088
  10   58.3  0.097
  11   57.8  0.107
  12   57.3  0.117
  13   56.8  0.126
  14   56.3  0.136
  15   55.8  0.145
  16   55.3  0.154
  17   54.8  0.163
  18   54.3  0.172
  19   53.8  0.180
  20   53.3  0.188
  21   52.8  0.196
  22   52.3  0.203
  23   51.8  0.210
  24   51.3  0.216   showered -> batch ruined

Each batch showers at its own level; this batch's level is unknown.

## Your probe run (optional, this message)

Before the scored batch you may run a probe on the same crystallizer, with the
same sigma_max, on a different batch (its own start temperature, target and
seed loading). It is the object `env`, already constructed:

    state, info = env.reset()                                  # starts a fresh
        probe batch
    state, reward, done, truncated, info = env.step(action)  # one step;
        `state` has the keys above
    env.failure_reason                                         # None, "shower",
        "seeds_dissolved" or "timeout"
    env.showered, env.done                                     # True once the
        batch has showered / has ended

`done` is returned by `env.step`; `env.done`, `env.showered` and
`env.failure_reason` are attributes of `env`, not keys of `state`. Drive it
however you like, as many batches as you want, within 30 seconds of compute.
Print what you learn: only your printed output reaches the next step, and
output beyond 2000 characters is cut off. The controller you write afterwards
has no access to `env`, to this code or to its variables.

## The final run (your controller)

Whether now or after a probe run, the controller is `def policy(state)`; after
a probe run you will receive your printed output first. It runs on the scored
batch from its first step. It is called once per step with the state dict and
returns the action. It may keep state between calls (module-level variables
are fine); it runs on one batch. Import only `math` and `random`. Read only
the state keys listed above.

## Format

Reply with one Python code block and nothing else. If it defines `def
policy(state)`, it is your controller: the scored batch runs with it and the
task ends. Anything else is probe code: it runs on `env`, its printed output
comes back to you, and you then write the controller.
```

The second user message, after a probe run: its printed output, then the instruction for the final run.

```
Output of your probe run:
SIGMA_MAX_LOW=0.18100 SIGMA_MAX_HIGH=0.18240

Now the final run. Reply with one Python code block and nothing else: the
controller, defining `def policy(state)`. It runs on the scored batch (the
same sigma_max as your probe run, a different batch) from its first step and
has no access to `env` or to your probe code. It is called once per step with
the state dict and returns the action. It may keep state between calls
(module-level variables are fine); it runs on one batch. Import only `math`
and `random`. Read only the state keys listed above.
```

References

[1] Z. Shao et al. *DeepSeekMath*. arXiv:2402.03300, 2024.

[2] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv:2503.14476, 2025.

[3] A. Kazemnejad et al. *VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment*. arXiv:2410.01679, 2024.

[4] Z. Wang et al. *RAGEN: Training Agents by Reinforcing Reasoning*. 2025.

[5] Z. Liu et al. *GEM: Reinforcement Learning for Generalist Models*. 2025.

[6] Y. Zhou et al. *ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL*. ICML, 2024.

[7] P. Wang et al. *Math-Shepherd*. ACL, 2024.

[8] L. Luo et al. *Improve Mathematical Reasoning in Language Models by Automated Process Supervision*. arXiv:2406.06592, 2024.

[9] G. Wang et al. *Voyager*. TMLR, 2024.

[10] B. Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2024.

[11] A. Novikov et al. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. 2025.

[12] C. Lu et al. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. arXiv:2408.06292, 2024.

[13] X. Luo et al. *Agent Lightning: Train ANY AI Agents with Reinforcement Learning*. arXiv:2508.03680, 2025.

[14] W. Zeng et al. *Reinforcing Multi-Turn Reasoning in LLM Agents via Turn-Level Reward Design*. arXiv:2505.11821, 2025.

[15] J. Oh et al. *Discovering State-of-the-art Reinforcement Learning Algorithms*. Nature, 2025.

[16] H. Le et al. *CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning*. NeurIPS, 2022.

[17] J. Liu et al. *RLTF: Reinforcement Learning from Unit Test Feedback*. 2023.

[18] Y. Wei et al. *SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution*. 2025.

[19] M. Jin et al. *ReVeal: Reinforcement Learning for Verifiable Multi-Turn Code Generation*. 2025.